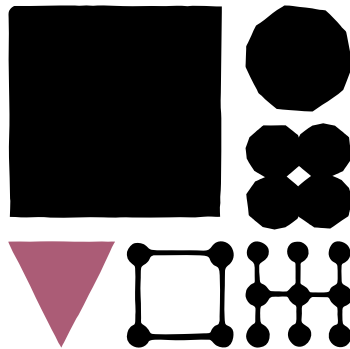


Claude 开发者平台的高级工具使用能力

Introducing advanced tool use on the Claude Developer Platform

Anthropic · 2025-11-24 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网页导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



我们新增了三项测试版功能，让 Claude 能够动态发现、学习和执行工具。下面介绍它们的工作方式。

未来的 AI Agent 将能够在数百乃至数千个工具之间无缝协作。例如，一个集成了 git 操作、文件处理、包管理器、测试框架和部署流水线的 IDE 助手；或者，一个同时连接 Slack、GitHub、Google Drive、Jira、公司数据库以及数十个 MCP 服务器的运营协调助手。

要构建高效的 Agent，就需要让它们能够使用规模不受限的工具库，而不必预先将每个工具定义都塞进上下文。我们关于结合 MCP 使用代码执行的博客讨论过：有时，Agent 还没读取用户请求，工具结果和工具定义就已消耗超过 50,000 个 token。Agent 应当按需发现和加载工具，只保留与当前任务相关的内容。

Agent 还需要能够从代码中调工具。使用自然语言进行工具调用时，每次调用都需要完整运行一轮模型推理，无论中间结果是否有用，它们都会不断堆积在上下文中。代码天然适合表达循环、条件判断和数据转换等编排逻辑。Agent 需要能够根据当前任务，灵活选择代码执行或模型推理。

Agent 还需要通过示例学习正确使用工具，而不只是依赖 schema 定义。JSON schema 定义了结构上什么是合法的，却无法表达使用模式：何时应该包含可选参数、哪些参数组合有意义，以及你的 API 期望遵循什么约定。

今天，我们发布了三项使这些能力成为可能的功能：

- **工具搜索工具 (Tool Search Tool)**，让 Claude 能够借助搜索工具访问数千个工具，而不占满上下文窗口。
- **程序化工具调用 (Programmatic Tool Calling)**，让 Claude 能够在代码执行环境中调用工具，减少对模型上下文窗口的影响。
- **工具使用示例 (Tool Use Examples)**，提供一种通用标准，用来展示如何有效使用某个工具。

在内部测试中，我们发现这些功能帮助我们构建了一些传统工具使用模式无法实现的东西。例如，**Claude for Excel** 使用程序化工具调用来读取和修改包含数千行数据的电子表格，而不会让模型的上下文窗口过载。

根据我们的经验，我们相信，这些功能为你使用 Claude 构建产品开辟了新的可能性。

工具搜索工具

面临的挑战

MCP 工具定义提供了重要上下文，但连接的服务器越多，相关 token 开销就会不断累积。考虑一个接入五台服务器的配置：

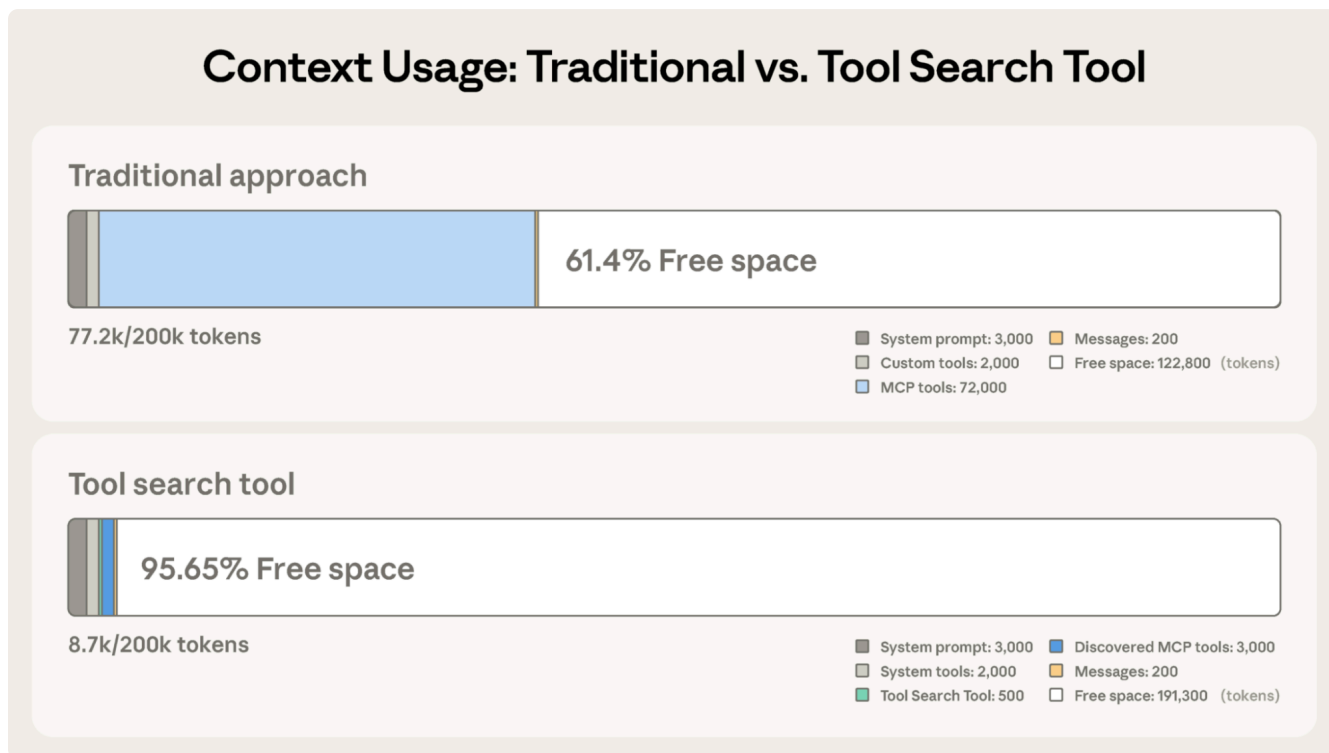
- GitHub：35 个工具（约 26K token）
- Slack：11 个工具（约 21K token）
- Sentry：5 个工具（约 3K token）
- Grafana：5 个工具（约 3K token）
- Splunk：2 个工具（约 2K token）

这意味着，对话还没有开始，58 个工具就消耗了约 55K token。再加上 Jira 等服务器（仅 Jira 就需要约 17K token），开销很快就会达到 100K token 以上。在 Anthropic，我们见过优化前的工具定义消耗 134K token 的情况。

但 token 成本并不是唯一问题。最常见的失败是选错工具和使用错误参数，尤其当工具名称相似时，例如 `notification-send-user` 和 `notification-send-channel`。

我们的解决方案

工具搜索工具不再预先加载所有工具定义，而是按需发现工具。Claude 只会看到当前任务实际需要的工具。



Tool Search Tool preserves 191,300 tokens of context compared to 122,800 with Claude's traditional approach.

传统方式：

- 预先加载所有工具定义（50 多个 MCP 工具约需 72K token）。
- 对话历史和系统提示词争夺剩余空间。
- 上下文总消耗：尚未开始任何工作，就已用掉约 77K token。

使用工具搜索工具后：

- 预先只加载工具搜索工具本身（约 500 token）。
- 按需发现工具（3 至 5 个相关工具，约 3K token）。
- 上下文总消耗：约 8.7K token，保留了 95% 的上下文窗口。

这样，在仍可访问完整工具库的同时，token 使用量减少了 85%。内部测试表明，在使用大型工具库时，MCP 评测的准确率显著提升。启用工具搜索工具后，Opus 4 从 49% 提高到 74%，Opus 4.5 从 79.5% 提高到 88.1%。

工具搜索工具如何工作

工具搜索工具让 Claude 能够动态发现工具，而不是预先加载所有定义。你将全部工具定义提供给 API，但通过 `defer_loading: true` 标记工具，使其可以按需发现。延迟加载的工具一开始不会进入 Claude 的上下文。Claude 只能看到工具搜索工具本身，以及设置为 `defer_loading: false` 的工具，也就是最关键、最常用的那些工具。

当 Claude 需要某项具体能力时，它会搜索相关工具。工具搜索工具返回匹配工具的引用，这些引用随后在 Claude 的上下文中展开为完整定义。

例如，如果 Claude 需要与 GitHub 交互，它会搜索“github”，随后只加载 `github.createPullRequest` 和 `github.listIssues`，而不会加载来自 Slack、Jira 和 Google Drive 的其他 50 多个工具。

这样，Claude 既能访问你的完整工具库，又只需为实际需要的工具承担 token 成本。

关于提示缓存：工具搜索工具不会破坏提示缓存，因为延迟加载的工具完全不包含在初始提示中。只有 Claude 搜索后，它们才会加入上下文，因此你的系统提示词和核心工具定义仍然可以被缓存。

实现方式：

```
{
  "tools": [
    // Include a tool search tool (regex, BM25, or custom)
    {"type": "tool_search_tool_regex_20251119", "name": "tool_search_tool_regex"},

    // Mark tools for on-demand discovery
    {
      "name": "github.createPullRequest",
      "description": "Create a pull request",
      "input_schema": {...},
      "defer_loading": true
    }
    // ... hundreds more deferred tools with defer_loading: true
  ]
}
```

对于 MCP 服务器，你可以对整台服务器启用延迟加载，同时保留特定的高频工具为预加载状态：

```
{
  "type": "mcp_toolset",
  "mcp_server_name": "google-drive",
  "default_config": {"defer_loading": true}, # defer loading the entire server
  "configs": {
    "search_files": {
      "defer_loading": false
    } // Keep most used tool loaded
  }
}
```

Claude 开发者平台提供开箱即用的、基于正则表达式和 BM25 的搜索工具，你也可以使用嵌入向量或其他策略实现自定义搜索工具。

何时使用工具搜索工具

和任何架构决策一样，启用工具搜索工具也涉及取舍。它会在工具调用前增加一步搜索，因此，当节省的上下文与准确率提升足以抵消额外延迟时，收益最大。

适合使用的情况：

- 工具定义消耗超过 10K token。

- 遇到工具选择准确率问题。
- 构建接入多台服务器、由 MCP 驱动的系统。
- 可用工具数量达到 10 个以上。

收益较小的情况：

- 工具库较小，工具少于 10 个。
- 每次会话中都会频繁使用全部工具。
- 工具定义本身很简短。

程序化工具调用

面临的挑战

随着工作流变得复杂，传统工具调用会带来两个根本问题：

- **中间结果污染上下文：**当 Claude 分析一个 10MB 日志文件以寻找错误模式时，整个文件都会进入上下文窗口，尽管 Claude 实际只需要错误频率的摘要。从多个表中获取客户数据时，每条记录都会在上下文中累积，不论是否相关。这些中间结果会消耗大量 token 预算，甚至可能把重要信息彻底挤出上下文窗口。
- **推理开销和人工式归纳：**每次工具调用都需要完整运行一轮模型推理。收到结果后，Claude 必须“目测”数据以提取相关信息，推理各部分如何联系，再决定下一步做什么，这一切都通过自然语言处理完成。一个涉及五个工具的工作流，就意味着五轮模型推理，再加上 Claude 逐项解析结果、比较数值和归纳结论的过程。这既慢，也容易出错。

我们的解决方案

程序化工具调用让 Claude 能够通过代码编排工具，而不是为每个工具单独进行一轮 API 往返。Claude 不必再逐个请求工具，并让每项结果都返回上下文；它可以编写代码，调用多个工具、处理输出，并控制哪些信息真正进入上下文窗口。

Claude 擅长编写代码。让它用 Python 表达编排逻辑，而不是通过自然语言逐项调用工具，就能获得更可靠、更精确的控制流。循环、条件判断、数据转换和错误处理，都会明确写在代码中，而不是隐含在 Claude 的推理里。

示例：预算合规检查

考虑一个常见业务任务：“哪些团队成员超出了第三季度差旅预算？”

你有三个可用工具：

- `get_team_members(department)`：返回团队成员列表，包含 ID 和职级。
- `get_expenses(user_id, quarter)`：返回某位用户的费用明细。
- `get_budget_by_level(level)`：返回某个员工职级的预算上限。

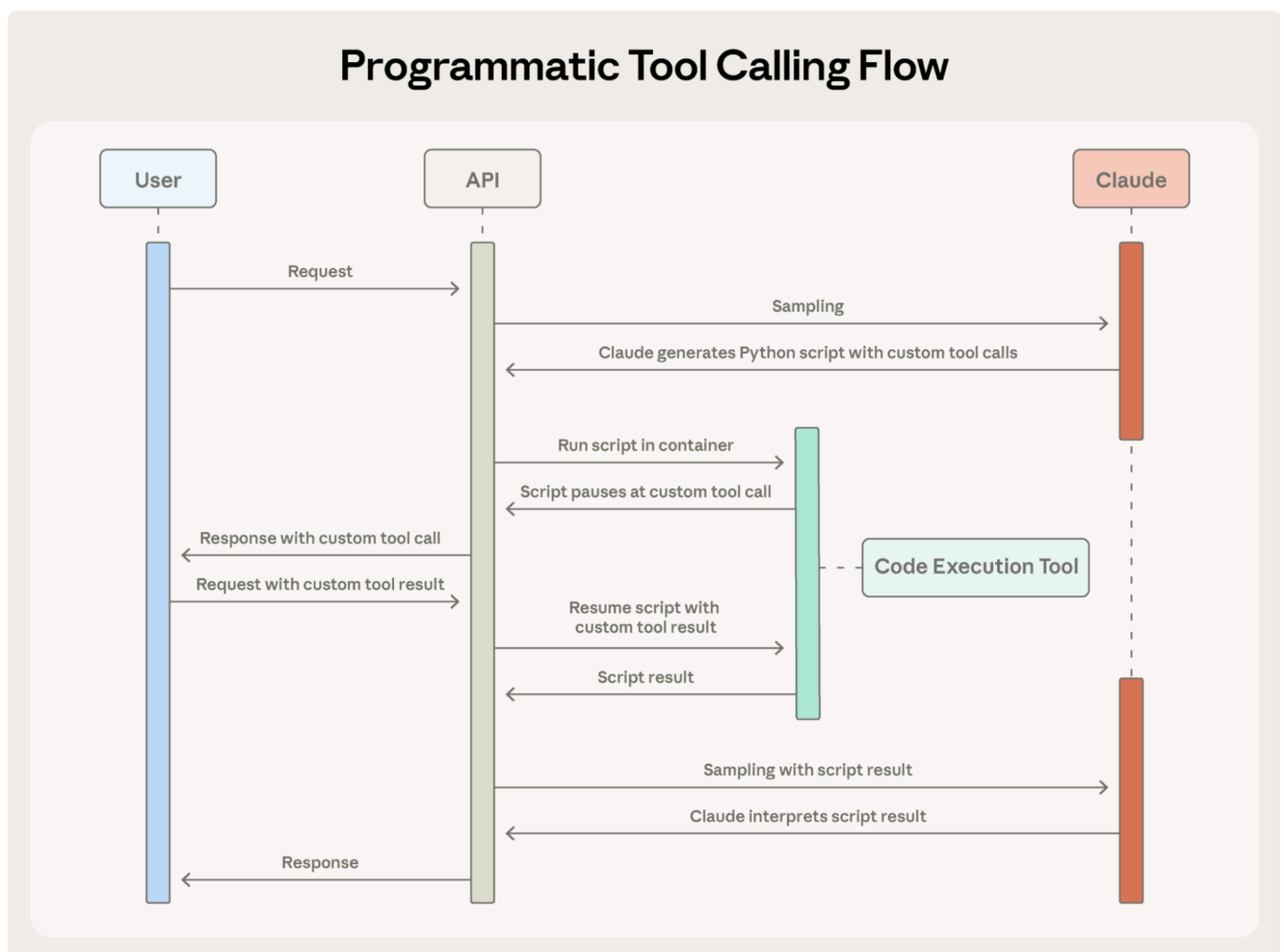
传统方式：

- 获取团队成员，共 20 人。

- 逐人获取第三季度费用，共 20 次工具调用，每次返回 50 至 100 条明细，包括航班、酒店、餐饮和收据。
- 按员工职级获取预算上限。
- 所有这些内容都进入 Claude 上下文：2,000 多条费用明细，超过 50 KB。
- Claude 以人工式方式汇总每个人的费用、查找其预算，并将费用与预算上限比较。
- 需要更多次模型往返，并大量消耗上下文。

使用程序化工具调用后：

Claude 编写一个 Python 脚本，编排整个工作流，而不是让每个工具结果都返回模型。脚本在代码执行工具提供的沙箱环境中运行，需要你的工具结果时便暂停。当你通过 API 返回工具结果后，结果由脚本处理，而不是由模型读取。脚本继续执行，Claude 只会看到最终输出。



Programmatic Tool Calling enables Claude to orchestrate tools through code rather than through individual API round-trips, allowing for parallel tool execution.

对于预算合规任务，Claude 编写的编排代码如下：

```
team = await get_team_members("engineering")

# Fetch budgets for each unique level
levels = list(set(m["level"] for m in team))
budget_results = await asyncio.gather(*[
    get_budget_by_level(level) for level in levels
])

# Create a lookup dictionary: {"junior": budget1, "senior": budget2, ...}
budgets = {level: budget for level, budget in zip(levels, budget_results)}

# Fetch all expenses in parallel
expenses = await asyncio.gather(*[
    get_expenses(m["id"], "Q3") for m in team
])

# Find employees who exceeded their travel budget
exceeded = []
for member, exp in zip(team, expenses):
    budget = budgets[member["level"]]
    total = sum(e["amount"] for e in exp)
    if total > budget["travel_limit"]:
        exceeded.append({
            "name": member["name"],
            "spent": total,
            "limit": budget["travel_limit"]
        })

print(json.dumps(exceeded))
```

Claude 的上下文只接收最终结果：超出预算的那两三个人。2,000 多条明细、中间求和结果，以及预算查询结果，都不会影响 Claude 的上下文，输入从 200KB 原始费用数据缩减为仅 1KB 的结果。

效率提升相当显著：

- **节省 token：**程序化工具调用（PTC）通过让中间结果留在 Claude 上下文之外，大幅减少 token 消耗。在复杂研究任务中，平均使用量从 43,588 token 降至 27,297 token，减少了 37%。
- **降低延迟：**每次 API 往返都需要模型推理，耗时从数百毫秒到数秒不等。当 Claude 在同一个代码块中编排 20 多次工具调用时，就省去了 19 次以上的模型推理。API 负责处理工具执行，无需每次都返回模型。
- **提高准确率：**通过编写明确的编排逻辑，Claude 比起用自然语言同时处理多个工具结果，出错更少。内部知识检索表现从 25.6% 提高到 28.5%；GIA 基准从 46.5% 提高到 51.2%。

生产工作流涉及杂乱数据、条件逻辑，以及需要扩展规模的操作。程序化工具调用让 Claude 能够通过程序处理这些复杂性，同时将注意力放在可据以行动的结果上，而不是原始数据处理上。

程序化工具调用如何工作

1. 将工具标记为可由代码调用

将 `code_execution` 加入 `tools`，并设置 `allowed_callers`，为选定工具启用程序化执行：

```
{
  "tools": [
    {
      "type": "code_execution_20250825",
      "name": "code_execution"
    },
    {
      "name": "get_team_members",
      "description": "Get all members of a department...",
      "input_schema": {...},
      "allowed_callers": ["code_execution_20250825"] # opt-in to programmatic tool calling
    },
    {
      "name": "get_expenses",
      ...
    },
    {
      "name": "get_budget_by_level",
      ...
    }
  ]
}
```

API 会将这些工具定义转换成 Claude 可以调用的 Python 函数。

2. Claude 编写编排代码

Claude 不再逐个请求工具，而是生成 Python 代码：

```
{
  "type": "server_tool_use",
  "id": "srvtoolu_abc",
  "name": "code_execution",
  "input": {
    "code": "team = get_team_members('engineering')\n..." # the code example above
  }
}
```


3. 工具执行不经过 Claude 上下文

当代码调用 `get_expenses()` 时，你会收到一个带有 `caller` 字段的工具请求：

```
{
  "type": "tool_use",
  "id": "toolu_xyz",
  "name": "get_expenses",
  "input": {"user_id": "emp_123", "quarter": "Q3"},
  "caller": {
    "type": "code_execution_20250825",
    "tool_id": "srvtoolu_abc"
  }
}
```

你提供结果后，结果会在代码执行环境中处理，而不会进入 Claude 的上下文。代码中的每次工具调用，都会重复这个请求—响应循环。

4. 只有最终输出进入上下文

代码运行结束时，只将代码的结果返回给 Claude：

```
{
  "type": "code_execution_tool_result",
  "tool_use_id": "srvtoolu_abc",
  "content": {
    "stdout": "[{\"name\": \"Alice\", \"spent\": 12500, \"limit\": 10000}...]"
  }
}
```

Claude 看到的就只有这些，而不是过程中处理的 2,000 多条费用明细。

何时使用程序化工具调用

程序化工具调用会为工作流增加一个代码执行步骤。当 token 节省、延迟改善和准确率提升足够显著时，这份额外开销就是值得的。

收益最大的情况：

- 处理大型数据集，但只需要聚合结果或摘要。
- 运行包含三个或更多相互依赖工具调用的多步骤工作流。
- 在 Claude 看到工具结果之前，对其进行过滤、排序或转换。
- 处理不应让中间数据影响 Claude 推理的任务。
- 针对大量对象并行操作，例如检查 50 个端点。

收益较小的情况：

- 进行简单的单工具调用。
- 任务要求 Claude 查看并推理所有中间结果。
- 进行返回内容较少的快速查询。

工具使用示例

面临的挑战

JSON Schema 擅长定义结构，包括类型、必填字段和允许的枚举值，但无法表达使用模式：何时包含可选参数、哪些组合有意义，以及你的 API 期望什么约定。

考虑一个客服工单 API：

```
{
  "name": "create_ticket",
  "input_schema": {
    "properties": {
      "title": {"type": "string"},
      "priority": {"enum": ["low", "medium", "high", "critical"]},
      "labels": {"type": "array", "items": {"type": "string"}},
      "reporter": {
        "type": "object",
        "properties": {
          "id": {"type": "string"},
          "name": {"type": "string"},
          "contact": {
            "type": "object",
            "properties": {
              "email": {"type": "string"},
              "phone": {"type": "string"}
            }
          }
        }
      },
      "due_date": {"type": "string"},
      "escalation": {
        "type": "object",
        "properties": {
          "level": {"type": "integer"},
          "notify_manager": {"type": "boolean"},
          "sla_hours": {"type": "integer"}
        }
      }
    },
    "required": ["title"]
  }
}
```

schema 定义了哪些输入合法，却没有回答以下关键问题：

- **格式歧义：** `due_date` 应该使用“2024-11-06”、“Nov 6, 2024”，还是“2024-11-06T00:00:00Z”？
- **ID 约定：** `reporter.id` 是 UUID、“USR-12345”，还是仅仅“12345”？
- **嵌套结构的用法：** Claude 何时应该填写 `reporter.contact` ？
- **参数关联：** `escalation.level` 和 `escalation.sla_hours` 与优先级之间有什么关系？

这些歧义可能导致格式错误的工具调用，以及不一致的参数使用方式。

我们的解决方案

工具使用示例允许你直接在工具定义中提供调用样例。你无需仅依赖 schema，而是可以向 Claude 展示具体的使用模式：

```
{
  "name": "create_ticket",
  "input_schema": { /* same schema as above */ },
  "input_examples": [
    {
      "title": "Login page returns 500 error",
      "priority": "critical",
      "labels": ["bug", "authentication", "production"],
      "reporter": {
        "id": "USR-12345",
        "name": "Jane Smith",
        "contact": {
          "email": "jane@acme.com",
          "phone": "+1-555-0123"
        }
      },
      "due_date": "2024-11-06",
      "escalation": {
        "level": 2,
        "notify_manager": true,
        "sla_hours": 4
      }
    },
    {
      "title": "Add dark mode support",
      "labels": ["feature-request", "ui"],
      "reporter": {
        "id": "USR-67890",
        "name": "Alex Chen"
      }
    },
    {
      "title": "Update API documentation"
    }
  ]
}
```

从这三个示例中，Claude 可以学到：

- **格式约定：**日期使用 YYYY-MM-DD，用户 ID 遵循 USR-XXXXX，标签使用 kebab-case，即以连字符连接单词。
- **嵌套结构模式：**如何构造 reporter 对象，以及其中嵌套的 contact 对象。
- **可选参数关联：**严重缺陷需要完整联系信息，以及带有严格 SLA 的升级处理设置；功能请求包含 reporter，但不包含 contact/escalation；内部任务则只需要标题。

在我们自己的内部测试中，工具使用示例将复杂参数处理的准确率从 72% 提升到了 90%。

何时使用工具使用示例

工具使用示例会增加工具定义中的 token，因此，当准确率提升超过额外成本时，它最有价值。

收益最大的情况：

- 复杂嵌套结构中，合法 JSON 并不意味着用法正确。
- 工具包含许多可选参数，而且何时包含哪些参数十分重要。
- API 存在 schema 未能表达的领域特定约定。
- 工具彼此相似，需要通过示例澄清该用哪一个，例如 `create_ticket` 与 `create_incident`。

收益较小的情况：

- 用法显而易见的简单单参数工具。
- URL 或电子邮件地址等 Claude 已经理解的标准格式。
- 适合通过 JSON Schema 约束来处理的验证问题。

最佳实践

构建能够在现实世界中执行操作的 Agent，意味着必须同时处理规模、复杂性和精确性。这三项功能可以协同解决工具使用 workflow 中的不同瓶颈。下面介绍如何有效组合它们。

有策略地叠加功能

并非每个 Agent 在执行某项任务时，都需要使用全部三项功能。先从最大的瓶颈入手：

- 工具定义导致上下文膨胀 → 工具搜索工具。
- 庞大中间结果污染上下文 → 程序化工具调用。
- 参数错误和格式不正确的调用 → 工具使用示例。

这种聚焦的方法，让你能够直接解决限制 Agent 表现的具体约束，而不是一开始就增加复杂度。

然后再按需叠加其他功能。它们彼此互补：工具搜索工具确保找到正确工具，程序化工具调用确保高效执行，工具使用示例确保正确调用。

配置工具搜索工具，改善工具发现

工具搜索会匹配名称和描述，因此清晰、具有描述性的定义可以提高发现准确率。

```
// Good
{
  "name": "search_customer_orders",
  "description": "Search for customer orders by date range, status, or total amount. Returns order details including items, shipping, and payment info."
}

// Bad
{
  "name": "query_db_orders",
  "description": "Execute order query"
}
```

在系统提示词中加入指导，让 Claude 知道有哪些能力可用：

```
You have access to tools for Slack messaging, Google Drive file management,
Jira ticket tracking, and GitHub repository operations. Use the tool search
to find specific capabilities.
```

让最常用的三至五个工具始终保持加载，其他工具延迟加载。这样，可以在常见操作的即时可用性与其余工具的按需发现之间取得平衡。

配置程序化工具调用，确保正确执行

由于 Claude 会编写代码来解析工具输出，应清楚记录返回格式。这有助于 Claude 写出正确的解析逻辑：

```
{
  "name": "get_orders",
  "description": "Retrieve orders for a customer.
Returns:
  List of order objects, each containing:
  - id (str): Order identifier
  - total (float): Order total in USD
  - status (str): One of 'pending', 'shipped', 'delivered'
  - items (list): Array of {sku, quantity, price}
  - created_at (str): ISO 8601 timestamp"
}
```

以下是适合选择启用程序化编排、能够从中受益的工具：

- 可以并行运行的工具，即彼此独立的操作。
- 可以安全重试的操作，即幂等操作。

配置工具使用示例，提高参数准确率

设计示例时，应让使用行为清晰明确：

- 使用贴近实际的数据，例如真实城市名称、合理价格，而不是“string”或“value”。

- 通过最少参数、部分参数和完整参数的不同写法展示多样性。
- 保持简洁：每个工具提供 1 至 5 个示例。
- 聚焦歧义：只有在无法从 schema 中明显看出正确用法时，才添加示例。

开始使用

这些功能目前以测试版提供。要启用它们，请添加 beta 请求头，并包含所需工具：

```
client.beta.messages.create(  
    betas=["advanced-tool-use-2025-11-20"],  
    model="claude-sonnet-4-5-20250929",  
    max_tokens=4096,  
    tools=[  
        {"type": "tool_search_tool_regex_20251119", "name": "tool_search_tool_regex"},  
        {"type": "code_execution_20250825", "name": "code_execution"},  
        # Your tools with defer_loading, allowed_callers, and input_examples  
    ]  
)
```

详细 API 文档和 SDK 示例，请参阅：

- 工具搜索工具的文档和实践示例。
- 程序化工具调用的文档和实践示例。
- 工具使用示例的文档。

这些功能推动工具使用从简单的函数调用走向智能编排。当 Agent 开始应对横跨数十种工具和大型数据集的更复杂工作流时，动态发现、高效执行和可靠调用，就成为基础能力。

我们期待看到你用它们构建什么。

致谢

本文由 Bin Wu 撰写，Adam Jones、Artur Renault、Henry Tay、Jake Noble、Noah Picard、Sam Jiang 和 Claude 开发者平台团队作出贡献。这项工作建立在 Chris Gorgolewski、Daniel Jiang、Jeremy Fox 和 Mike Lambert 的基础研究之上。我们也从整个 AI 生态中汲取了灵感，包括 Joel Pobar 的 LLMVM、Cloudflare 的 Code Mode，以及以代码执行方式使用 MCP。特别感谢 Andy Schumeister、Hamish Kerr、Keir Bradwell、Matt Bleifer 和 Molly Vorwerck 的支持。